

Service Discovery: Eureka and Consul

1. What is Service Discovery?

In a microservices system, services are often deployed as **multiple instances with dynamic IP addresses/ports**.

Suppose:

```
Payment Service

Instance 1 → 10.0.1.10:8080
Instance 2 → 10.0.1.11:8080
Instance 3 → 10.0.1.12:8080
```

Your BPS service should not hard-code:

```
http://10.0.1.10:8080
```

because instances can be:

- added
- removed
- restarted
- moved
- scaled dynamically

Instead, it asks a **service registry**:

```
BPS
↓
"Where is Payment Service?"
↓
Service Registry
↓
10.0.1.10
10.0.1.11
10.0.1.12
```

That is **service discovery**.

2. Two Types of Service Discovery

Client-Side Discovery

The calling service contacts the registry and chooses an instance.

```
BPS
↓
Eureka
↓
Payment instances
```

```
↓
choose one
↓
Payment Service
```

Historically, **Netflix Eureka** is commonly associated with this model.

Server-Side Discovery

The client sends the request to a load balancer/service router, and **that component** performs discovery.

```
BPS
↓
Load Balancer
↓
Service Registry / platform discovery
↓
Payment Service
```

This is common in container/orchestrated environments.

3. Eureka

Netflix Eureka is a service registry commonly used with Spring Cloud applications.

Architecture:



Each service registers itself.

For example:

```
Payment Service
↓
REGISTER
↓
Eureka Server
```

It provides information such as:

```
SERVICE: PAYMENT-SERVICE
HOST: 10.0.1.10
PORT: 8080
STATUS: UP
```

4. Eureka Registration

Conceptually:

```
Payment Service starts
    ↓
Connects to Eureka
    ↓
Registers itself
    ↓
Eureka stores instance information
```

Then BPS can discover it.

```
BPS
  ↓
Eureka
  ↓
PAYMENT-SERVICE
  ↓
instance list
```

5. Eureka Heartbeats

How does Eureka know an instance is still alive?

The client periodically sends a **heartbeat/renewal**.

```
Payment Service
  ↓
heartbeat
  ↓
Eureka
```

If the instance stops renewing and is determined to be unavailable, Eureka can remove/evict it according to its configuration and operating conditions.

This is called **lease/heartbeat-based registration**.

6. Eureka Client

A Spring Cloud application can act as an Eureka client.

Historically, Spring Boot applications used annotations such as:

```
@EnableDiscoveryClient
```

Modern Spring Cloud can often auto-detect discovery support, so you may not need to explicitly use that annotation depending on the setup/version.

The application is configured with the Eureka server location and registers itself.

For example, conceptually:

```
spring:
  application:
    name: payment-service

eureka:
```

```
client:
  service-url:
    defaultZone: http://localhost:8761/eureka/
```

Then:

```
payment-service
  ↓
Eureka Server
```

7. Calling a Service by Name

Without discovery:

```
restTemplate.getForObject(
    "http://10.0.1.10:8080/payments",
    ...
);
```

Bad for dynamic environments.

With discovery/load-balancing:

```
payment-service
```

becomes the logical service name.

Conceptually:

```
BPS
  ↓
payment-service
  ↓
Eureka
  ↓
instance selection
  ↓
Payment instance
```

8. Load Balancing

Suppose Eureka knows:

```
Payment Service
├─ Instance 1
├─ Instance 2
└─ Instance 3
```

BPS needs one instance.

A client-side load-balancing mechanism can choose one:

```
Request 1 → Instance 1
Request 2 → Instance 2
```

In Spring Cloud, **Spring Cloud LoadBalancer** is the modern load-balancing component commonly used instead of older Netflix Ribbon integrations.

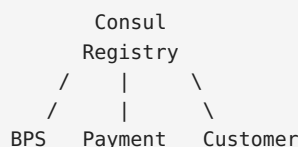
9. Consul

HashiCorp Consul is more than just service discovery.

It provides capabilities such as:

- service discovery
- health checking
- key-value configuration
- DNS-based discovery
- service networking capabilities in broader Consul environments

Basic model:

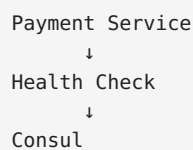


Services register with Consul and advertise their health and location.

10. Consul Health Checks

One important Consul feature is health checking.

Conceptually:



Suppose:

```
Instance 1 → healthy
Instance 2 → unhealthy
Instance 3 → healthy
```

Discovery can avoid unhealthy instances depending on how the client/load-balancing mechanism is configured.

11. Eureka vs Consul

Feature	Eureka	Consul
---------	--------	--------

Service discovery	Yes	Yes
Health checking	Heartbeat/lease model	Strong built-in health-check model
Spring integration	Strong	Strong
Key-value store	No core equivalent	Yes
DNS discovery	Not its primary model	Yes
Ecosystem	Netflix/Spring Cloud heritage	HashiCorp ecosystem
Typical use	Spring-centric systems	Broader platform/service discovery

12. Important Modern Architecture Point

For a **modern Kubernetes-based deployment**, you may not need Eureka or Consul for basic service discovery.

Kubernetes provides built-in service discovery through **Services and DNS**.

For example:

```
BPS
↓
http://payment-service
↓
Kubernetes Service
↓
Payment Pods
```

So in an interview, don't say:

“Every microservice architecture needs Eureka.”

That's incorrect.

A better statement:

“A service registry such as Eureka or Consul is one approach to service discovery. In Kubernetes, the platform itself commonly provides service discovery through Service objects and DNS.”

13. Eureka vs Kubernetes Discovery

Traditional Spring Cloud architecture

```
BPS
↓
Eureka
↓
Payment instances
```

Kubernetes architecture

```
BPS Pod
↓
```

```
payment-service
↓
Kubernetes Service
↓
Payment Pods
```

This is important because interviewers increasingly expect candidates to distinguish **application-level discovery** from **platform-provided discovery**.

14. What Problem Does Service Discovery Solve?

Without discovery:

```
BPS
↓
hard-coded IP
↓
Payment Service
```

Problems:

```
Instance IP changes
Scaling
Failover
Dynamic deployment
```

With discovery:

```
BPS
↓
service name
↓
discovery mechanism
↓
healthy instance
```

So the key benefit is:

Decoupling service consumers from the physical network location of service instances.

15. What Happens When an Instance Goes Down?

Suppose:

```
Payment Service
├─ P1 ✓
├─ P2 ✗
└─ P3 ✓
```

Discovery/health-check infrastructure identifies P2 as unavailable.

Then traffic should be directed toward healthy instances according to the configured discovery/load-balancing behavior.

```
BPS
↓
Discovery
↓
P1 / P3
```

But remember:

Service discovery itself isn't the same thing as complete fault tolerance.

You still need:

- timeouts
- retry strategy
- circuit breakers
- idempotency
- monitoring

16. Service Discovery vs Load Balancing

Don't confuse these.

Service Discovery

Answers:

Where are the available instances?

Load Balancing

Answers:

Which instance should receive this request?

They often work together:

```
Discovery
↓
Instance list
↓
Load Balancer
↓
Selected instance
```

17. Service Registry vs API Gateway

Another common interview distinction.

Service Registry

Used by services/platform components to discover other services.

BPS → Registry → Payment Service

API Gateway

Usually the entry point for external clients.

```
Frontend
  ↓
API Gateway
  ↓
BPS / Payment / Customer
```

An API Gateway may itself use service discovery behind the scenes.

18. What If Eureka Goes Down?

This is a good interview question.

A simplistic answer:

“All microservices stop working.”

Not necessarily.

In Eureka-based systems, clients can cache registry information and continue using known instances for some time, depending on the configuration and current state.

But **new registrations/discoveries can be affected**, and stale instance information can become a problem.

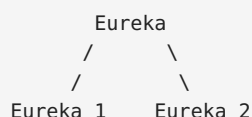
The broader lesson:

The service registry can become an important infrastructure dependency, so it needs appropriate availability, replication and operational planning.

19. Eureka Server High Availability

You shouldn't normally run an important production registry as one isolated instance.

You can have:



Clients can be configured to communicate with multiple registry peers/endpoints according to the architecture.

20. Self-Registration

One of the most important concepts:

```
Service starts
↓
Registers with registry
↓
Sends heartbeat/health information
↓
Registry knows where it is
```

This removes the need for manually maintaining a central list of service instances.

21. Practical BPS Example

Imagine your architecture is:

```
API Gateway
↓
BPS
↓
Payment Service
↓
External Gateway
```

There are three Payment Service instances:

```
Payment Service
├── 10.0.1.10
├── 10.0.1.11
└── 10.0.1.12
```

BPS should not know those IPs permanently.

Instead:

```
BPS
↓
"payment-service"
↓
Eureka / Consul / Kubernetes
↓
healthy instances
↓
Load Balancer
↓
Payment instance
```

That is the story you should understand.

22. Interview Questions

Basic

1. What is service discovery?

2. Why do microservices need service discovery?
3. What is a service registry?
4. What is Eureka?
5. What is Consul?
6. How does a service register itself?
7. How does another service discover it?

Intermediate

8. What are heartbeats?
9. What happens if a service instance goes down?
10. How does load balancing work with service discovery?
11. Service discovery vs load balancing?
12. Service discovery vs API Gateway?
13. Client-side vs server-side discovery?
14. What happens if the registry is unavailable?
15. How do you achieve registry high availability?

3-year level

16. Eureka vs Consul?
17. How does Consul health checking work?
18. Why would you choose Consul over Eureka?
19. Do you need Eureka when using Kubernetes?
20. How does Kubernetes service discovery work?
21. How would you debug “service not found”?
22. How would you prevent calls to unhealthy instances?
23. How do timeout/retry/circuit breaker interact with discovery?

23. Very Likely Interview Scenario

Interviewer:

“BPS calls Payment Service. Payment Service has three instances. One instance goes down. How does BPS know where to send the request?”

Answer:

“With a service-discovery mechanism, Payment Service instances register their locations with the registry and periodically maintain their liveness information. BPS resolves the logical service name to available instances, and the load-balancing layer selects an instance. If one instance becomes unavailable and is removed from the available instance set, requests can be routed to healthy instances. The exact mechanism depends on whether we’re using Eureka, Consul, Kubernetes Service/DNS, or another platform.”

That’s much better than:

“Eureka automatically sends the request to another server.”

24. One Important Distinction for Your Interview

You may encounter this architecture:

```
Client
↓
API Gateway
↓
BPS
↓
Payment Service
↓
Database
```

The interviewer could ask:

“Why do you need Eureka if you already have an API Gateway?”

Answer:

“They solve different problems. The API Gateway handles external entry-point concerns such as routing and policy enforcement, while service discovery allows internal components to locate service instances dynamically. The gateway itself can also use service discovery.”

25. Interview-Ready Answer

“Service discovery solves the problem of locating dynamically changing service instances in a microservices environment. Instead of hard-coding IP addresses, services register themselves with a discovery mechanism and consumers resolve a logical service name to available instances. Eureka is a service registry commonly associated with Spring Cloud and Netflix’s ecosystem, while Consul provides service discovery along with health checks, DNS-based discovery and key-value capabilities. Discovery and load balancing are related but different: discovery provides the available instances, while load balancing selects an instance. In Kubernetes, built-in Service and DNS mechanisms often remove the need for a separate Eureka server for basic service discovery.”

Memorize this:

```
Service Discovery
→ "Where is the service?"

Load Balancing
→ "Which instance should I call?"

API Gateway
→ "How do external clients enter the system?"
```

For your **BPS project**, the important next concept is **inter-service communication: REST/Feign/WebClient vs Kafka**, because an interviewer can naturally go:

Microservices → Service Discovery → Load Balancing → Feign → Timeout → Retry → Circuit Breaker → Failure handling.